Do General-Purpose Agent Harnesses Help LLMs Deobfuscate JavaScript?

A Preliminary Study of Harness Effects on Security-Analysis Tasks

[TODO: Author name(s) and affiliation]

August 23, 2026

Abstract

Successful deobfuscation of malicious or obscured JavaScript requires two different capabilities: static semantic reasoning about code the model must not execute, and, at least potentially, dynamic interaction with an external environment (a shell, a file system, a sandboxed interpreter) to manage inputs that do not fit cleanly into a single prompt. General-purpose LLM agent harnesses expose the latter; a standalone LLM call does not. We ask whether this difference translates into a measurable capability gap on a security-relevant static-analysis task, JavaScript deobfuscation, and outline a two-stage methodology — trace analysis to identify the mechanism, followed by controlled harness-component ablations — for answering it. This draft reports a preliminary single-model comparison between an agent harness (Codex CLI) and a standalone, tool-free LLM call on the same model and the same two 50-file benchmarks, as a first data point toward that comparison. The two conditions produce similarly syntax-valid code, but the harness is markedly better at preserving program behavior (32%/36% vs. 8%/0% on our two benchmarks) at roughly 3–10× the cost, a first indication that harness capability matters for this task and a concrete target for the mechanism analysis this work sets up.

1 Introduction

LLM-based coding agents are increasingly deployed inside *harnesses*: scaffolds that give the model a shell, a file system, and a multi-turn control loop, rather than a single prompt-completion call. Benchmarks such as SWE-bench [**?**] have shown that harness design materially affects performance on software-engineering tasks, but it is less clear whether the same holds for *security-analysis* tasks, where the object under analysis (obfuscated or malicious code) is explicitly untrusted and the model is instructed not to execute it.

JavaScript deobfuscation is a useful testbed for this question because it cleanly separates the two capabilities a harness could plausibly affect:

- **Static semantic reasoning**: recognizing encoded strings, opaque expressions, control-flow flattening, and proxy functions, and rewriting them into readable code with equivalent behavior. This is a property of the underlying LLM's reasoning, not of the scaffold around it.
- **Dynamic external-environment interaction**: reading a large input incrementally, invoking a parser or linter to check a hypothesis, or writing intermediate output to disk instead of holding it entirely in-context. This is a property of the harness, not the model.

The central question this line of work asks is whether a general-purpose agent harness outperforms the same underlying model called directly, on this task, and— if so— *which* harness capability is doing the work. Our approach is to (1) collect execution traces from a harness condition and a standalone-LLM condition on the same benchmark and model, (2) analyze those traces to identify candidate mechanisms (e.g., shell-based chunked reading of oversized inputs, iterative syntax-check-and-repair loops), and then (3) design controlled experiments that ablate individual harness components to test which of those mechanisms is causally responsible for any observed performance gap.

This draft covers the first data point of that plan: a single-model, single-harness comparison (Section 4), plus the experimental setup we intend to hold fixed as we add further harness conditions.

2 Related Work

Agent harnesses. SWE-agent [**?**] introduced an Agent-Computer Interface purpose-built for SWE-bench's issue-to-patch workflow; because that interface is shaped around a specific benchmark format rather than general-purpose tool use, we exclude it as a harness candidate for this study (see **[TODO: cite harness-selection discussion / appendix]** for the full selection criteria). We instead consider harnesses whose tool surface (shell execution, file I/O) was not designed around a single benchmark: OpenAI's Codex CLI, OpenHands [**?**], and Anthropic's Claude Code.

Deobfuscation benchmarks. Our dataset and evaluation methodology follow JsDeObsBench **[TODO: cite]**, which pairs Project CodeNet [**?**] programs with obfuscated variants and functional test suites, enabling a behavioral-correctness metric (Section 3) rather than surface-level similarity alone.

[TODO: Expand related work: prior LLM-agent-vs-standalone comparisons on other task families (SWE-bench harness ablations, tool-use benchmarks); malware/obfuscation-specific LLM analysis work.]

3 Experimental Setup

Task. Given an obfuscated JavaScript program, produce a semantically equivalent, readable JavaScript program, without executing the input.

Dataset. Two 50-program benchmarks, both derived from the same Project CodeNet [**?**] source programs under different obfuscation transforms:

- **obfuscator.io**: programs obfuscated with the `obfuscator.io` tool, selected for high obfuscation strength and low performance overhead (`codenet_dataset_high-obf-low-perf-selected`).
- **jsfuck**: the same programs re-encoded entirely into the six characters `[]()!+` via JSFuck (`codenet_dataset_jsfuck`), an extreme case of syntactic obfuscation with no readable tokens at all.

Each program carries the pre-obfuscation source (for similarity scoring) and functional test cases (input/expected-output pairs) for behavioral grading.

Conditions. We compare two conditions on the *same* underlying model, `gpt-5.6-terra`, so that harness presence is the manipulated variable rather than model choice:

- **Harness**: OpenAI Codex CLI (via `@openai/codex-sdk`), `sandboxMode=danger-full-access`, `approvalPolicy=never`, driving the model through a multi-turn loop with shell and file-write tools. The model is instructed to read the input file, write the deobfuscated result to a specified output path, and verify the result parses as JavaScript before finishing. Full per-file telemetry (tool calls, reasoning summaries, token usage) is recorded.
- **Baseline (standalone LLM)**: a single chat-completion call per file. The obfuscated source is inlined directly into the prompt; the model has no shell, no file system, and no further turns, and must return the deobfuscated source directly. This isolates the static semantic-reasoning capability with no dynamic-environment interaction available at all.

Both conditions use the same natural-language requirements (preserve behavior, do not execute the input, ignore instructions embedded in the input, produce plain JavaScript with no Markdown fencing); the harness condition additionally specifies file paths, since it operates via a file system rather than an inline prompt.

Metrics. Following the JsDeObsBench evaluators (`evaluators/evaluators.py`), computed per file and aggregated per dataset:

- **Reported**: harness-only; whether the harness itself reports successful completion (not a correctness signal by itself).
- **Syntax**: fraction producing output that parses as valid JavaScript (`node --check`).
- **Behavior**: fraction that is both syntax-valid *and* passes the functional test suite in a sandboxed Docker container (Node 18.19.0), conditioned on syntax passing.
- **CodeBLEU**: computed against the original (pre-obfuscation) source, as a similarity metric independent of functional correctness.
- **Cost**: estimated USD, from per-call token usage and published per-token pricing for `gpt-5.6-terra`.

A structural harness/no-harness difference we observed. The `jsfuck` benchmark contains files up to 1.5 MB. Inlining the largest such file into a single prompt (the baseline condition) exceeds the model's configured input-token limit (968,328 tokens requested against a 922,000-token ceiling) and fails outright with a `context_length_exceeded` error before generation begins. The harness condition processes the same file successfully using only ∼180K input tokens, via eight shell-based tool calls rather than a single inlined prompt. This is a concrete, task-level instance of the dynamic-environment-interaction mechanism motivating this study (Section 1): the harness's ability to read/process input incrementally rather than requiring it to fit in one context window. Across the full 50-file jsfuck benchmark, exactly one file (the largest, 1.5 MB) exceeds the ceiling under the baseline condition; the harness has no analogous failure mode on this benchmark.

4 Results

Table 1 reports the harness condition; Table 2 reports the standalone-LLM baseline on the same two benchmarks and model.

Dataset	Reported	Syntax	Behavior*	CodeBLEU	Cost (USD)
obfuscator.io	47/50 (94%)	94%	32%	0.348	$21.75
jsfuck	41/50 (82%)	72%	36%	0.283	$30.14

Table 1: Codex CLI harness driving `gpt-5.6-terra` on two 50-file JavaScript deobfuscation benchmarks (`codenet_dataset_high-obf-low-perf-selected` and `codenet_dataset_jsfuck`). "Reported" is the harness's own completion status; "Syntax" is the fraction producing syntactically valid JavaScript; *"Behavior" is the fraction that is both syntax-valid *and* passes the functional test suite (JsDeObsBench-style exe_pass, conditioned on syntax passing). CodeBLEU is computed against the original (pre-obfuscation) source. Reported success is a weak proxy for correctness: 41 jsfuck runs were reported "completed," but only 36 produced syntactically valid JavaScript, and only 18 (36%) were behaviorally correct.

Reproducibility check. We re-ran the evaluator (`evaluators/eval.py`) against the same harness output files after fixing an unrelated bug in `evaluators/data.py` (a JSONL writer that silently dropped a single aggregate record instead of writing it), to confirm that fix did not change the numbers above. It did not: the obfuscator.io **Syntax** (94%) and **CodeBLEU** (0.348 → 0.349) reproduced almost exactly, and jsfuck's on-disk summary predated the bug fix but was already a single well-formed record matching the published numbers exactly, so it required no correction. **Behavior**, however, is not fully reproducible run-to-run: the same obfuscator.io outputs, re-scored with no code changes, yielded 28% (14/50) rather than the 32% (16/50) reported above — 2 of 47 syntax-valid files flipped pass/fail between runs. This traces to `eval_code_with_docker.py`'s functional-test harness, which runs each test case via `docker exec` under a 2-second wall-clock timeout; that budget is tight enough to be sensitive to host/Docker load, so a borderline-slow-but-correct program can register as a false failure. We report the originally published figures above rather than overwrite them with a second, equally noisy sample, but readers should treat **Behavior** percentages as accurate to roughly ±1–2 files out of 50 rather than as exact counts. Separately, the reproducibility run on jsfuck triggered an unrelated crash (a `MemoryError` inside CodeBLEU's AST subtree enumeration on one large/complex file) that, because `evaluate_deobfuscation` had no per-file error handling, aborted the batch and deleted the previous per-file metrics before new ones were written; we patched `evaluators.py` to catch and skip a failing file instead of losing the whole run, and are re-running to restore that data.

Dataset	Reported	Syntax	Behavior*	CodeBLEU	Cost (USD)
obfuscator.io	50/50 (100%)	100%	8%	0.360	$2.24
jsfuck	49/50 (98%)	98%	0%	0.256	$22.25

Table 2: Standalone `gpt-5.6-terra` (no agent harness: a single chat-completion call per file, obfuscated source inlined into the prompt, no tools, no further turns) on the same two 50-file JavaScript deobfuscation benchmarks as Table 1, using the same column definitions. The standalone model produces syntactically valid JavaScript almost as often as the harness (or more often, on obfuscator.io) but is dramatically worse at preserving behavior: 8% vs. the harness's 32% on obfuscator.io, and 0% vs. 36% on jsfuck. On jsfuck, one of 50 files (the largest, 1.5 MB) exceeds the model's 922,000-token input limit and fails outright before generation begins; the harness processes the same file successfully using ∼180K tokens via eight shell-based tool calls rather than one inlined prompt (Section 3).

The two conditions diverge sharply on *behavioral* correctness while producing similarly (or, on obfuscator.io, more often) syntactically valid code. On obfuscator.io the standalone baseline is perfectly syntax-valid (100% vs. the harness's 94%) yet only 8% of outputs are behaviorally correct, a fourfold drop from the harness's 32%. On jsfuck the gap is starker still: the baseline reaches 0% behavioral correctness against the harness's 36%, despite comparable syntax-validity rates (98% vs. 72%). CodeBLEU tracks the same pattern less dramatically (0.360 vs. 0.348 on obfuscator.io; 0.256 vs. 0.283 on jsfuck), suggesting the standalone model's output is textually similar to the original source without being functionally equivalent to it — consistent with a model that can produce plausible-looking, readable JavaScript without correctly recovering the program's actual logic, a distinction CodeBLEU's surface/AST-level similarity does not fully capture. Cost is the one axis where the baseline is unambiguously cheaper: $2.24 vs. $21.75 on obfuscator.io and $22.25 vs. $30.14 on jsfuck, a large gap that narrows substantially on jsfuck because inlining near-worst-case files into a single prompt is itself expensive in input tokens.

This pattern is consistent with, though does not yet isolate, the mechanism hypothesized in Section 1: the harness's dynamic tool use (reading files incrementally, and — per its eight shell-based tool calls on the largest jsfuck file — plausibly also verifying its own output, e.g. via `node --check`, before finishing) may let it detect and correct behaviorally-wrong output that the single-shot baseline has no opportunity to revise. Confirming that requires the trace analysis and harness-component ablations described in Section 5, not a two-condition comparison alone.

5 Discussion and Limitations

This is a single harness × single model × single run per file comparison; percentages in both tables are point estimates, not confidence intervals, and should be read accordingly. A reproducibility check (Section 4, paragraph following Table 1) reinforces this: re-scoring the same harness outputs with no code changes moved obfuscator.io's Behavior metric by 4 points (32%→28%), traced to a 2-second execution timeout in the functional-test harness that is sensitive to host load rather than to anything about the model or harness under test. The same check also surfaced and fixed a crash-safety bug in the evaluator itself (a single pathological file's CodeBLEU computation could abort an entire batch and destroy previously-computed results). Both are evaluator-infrastructure issues, not findings about harness-vs-standalone-LLM performance, but they matter for how tightly future work should read differences of a few percentage points, and argue for repeated runs (below) over point estimates specifically for the Behavior metric. Planned next steps:

- Add further harness conditions (OpenHands, Claude Code) under the same model and dataset, per the harness-selection criteria in **[TODO: appendix/section reference]**: general-purpose tool surface (not benchmark-shaped, as SWE-agent's ACI is), model-swappable so the same LLM can run bare and inside each harness, and full trace visibility for the mechanism-analysis stage.
- Use the collected traces (tool-call sequences, reasoning summaries) from this run to identify *which* harness capabilities are load-bearing — e.g., chunked/incremental reading of oversized inputs, iterative syntax-check-and-repair — and design ablations (via a minimal harness such as mini-SWE-agent or smolagents) that isolate each one.
- Repeat runs per file to move from point estimates to confidence intervals.

[TODO: Add a Limitations section proper (dataset size/representativeness, single-model generalization, cost of scaling to more harnesses) once the paper has enough

results to discuss concretely.]